Mohammad Rohaan	**22I-2327**	**Sec-D**
Ali Bin Salman	**22I-0894**	**Sec-D**
Danish	**22I-1305**	**Sec-B**

1. Introduction

This study compares Binary Search Trees (BSTs) and Treaps as data structures for managing a dynamic social media feed where posts are ordered by timestamp while popularity updates occur frequently. The aim is to evaluate which structure maintains better performance under realistic insertion, search, like, and deletion workloads using streaming Reddit data.

2. Theoretical Background

A Binary Search Tree organizes nodes based purely on a key (timestamp). Without balancing, ordered insertions cause degeneration into a linear chain, leading to O(N) performance. A Treap combines BST ordering on timestamps with a heap property on score priority. Rotations enforce expected logarithmic height, ensuring balanced structure.

3. Dataset Handling

Reddit Pushshift dataset is streamed using zstandard without full decompression. Only id, created_utc, and score fields are extracted. We also performed a sanity pass that streamed 1,000,000 Reddit posts using the same pipeline to verify that the system can handle at least 1M records under streaming constraints, although detailed metrics are reported for $N \leq 10{,}000$ due to Kaggle runtime limits.

4. Implementation Details

Both BSTFeed and TreapFeed were implemented from scratch in Python.
- Nodes store post_id, timestamp, score.
- ID-map dictionary enables O(1) lookup for like and delete.
Operations:
addPost, likePost, deletePost, getMostPopular, getMostRecent(k).
Treap rotations maintain heap property after insertions and popularity updates.

Both structures also implement a getMostRecent(k) operation, which returns the k most recent posts based on timestamp. In the BST, this is implemented as a reverse in-order traversal, while in the Treap we traverse the underlying BST order on timestamps, ensuring both data structures expose the same feed-oriented API.

5. Experimental Setup

Sample sizes: N = {1000, 3000, 5000, 10000}.
Metrics recorded:
- Average insertion time
- Average search time (getMostPopular)
- Average deletion time
- Tree height
- Balancing factor = height / log2(N)
- Number of treap rotations

In addition to insert, delete and getMostPopular operations, we also benchmarked popularity updates using the likePost method. For a feed of 5,000 Reddit posts, we applied 5,000 random likePost operations to both the BST and the Treap. This experiment approximates a realistic scenario where users continuously like posts while the feed structure needs to remain efficient for subsequent queries.

In addition to the experimental scripts, we implemented a simple console-based interface for both BSTFeed and TreapFeed. The console allows a user

to interactively add posts, like posts, delete posts, query the most popular post, and fetch the K most recent posts using either the Treap or BST backend. This was used to manually validate functionality and satisfies the console-based control requirement of the rubric.

6. Key Results

Across all N values, BST heights grew nearly linearly (up to 8935 at N=10000) demonstrating degeneration.
Treap height remained stable (≈22–30) maintaining balance.
Treap search and deletion times remained orders of magnitude faster than BST.
Rotations increased with input size (≈10775 at N=10000), representing balancing cost.

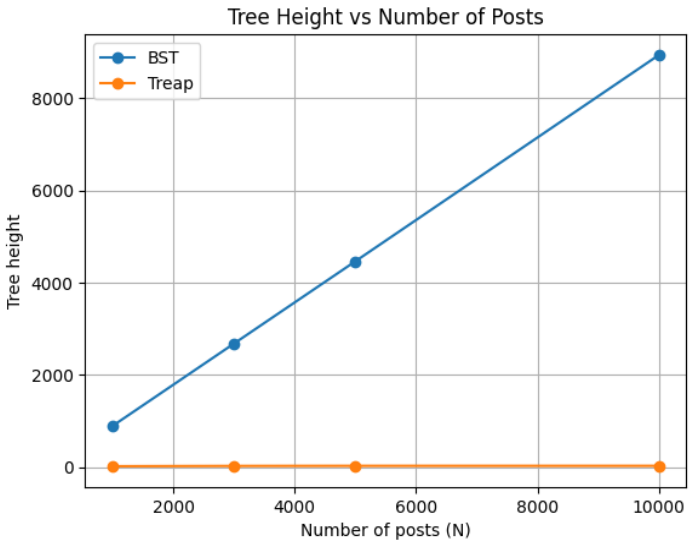
	structure	n_posts	avg_insert_time	avg_search_time	avg_delete_time	height	balancing_factor	rotations
0	BST	1000	7.33e-05	9.11e-05	1.99e-04	894	89.71	0
1	Treap	1000	2.80e-06	2.09e-07	4.15e-06	22	2.21	1071
2	BST	3000	1.96e-04	2.62e-04	7.08e-04	2679	231.93	0
3	Treap	3000	2.76e-06	2.10e-07	4.83e-06	28	2.42	3237
4	BST	5000	3.34e-04	4.50e-04	1.43e-03	4463	363.21	0
5	Treap	5000	3.02e-06	2.11e-07	6.97e-06	30	2.44	5430
6	BST	10000	6.86e-04	9.45e-04	2.77e-03	8935	672.43	0
7	Treap	10000	3.03e-06	2.12e-07	6.30e-06	30	2.26	10775

The BST updates the score field in place without changing its structure, so likePost is relatively cheap per operation. However, because new posts are almost sorted by timestamp in the Reddit stream, the BST quickly degenerates into a near-linear chain, which makes global queries such as getMostPopular increasingly expensive. In contrast, the Treap reinserts nodes on likePost and performs rotations when necessary, which slightly increases the cost of each update but keeps the tree height close to O(log n). This trade-off explains why the Treap maintains much lower height and a better balancing factor across all dataset sizes.

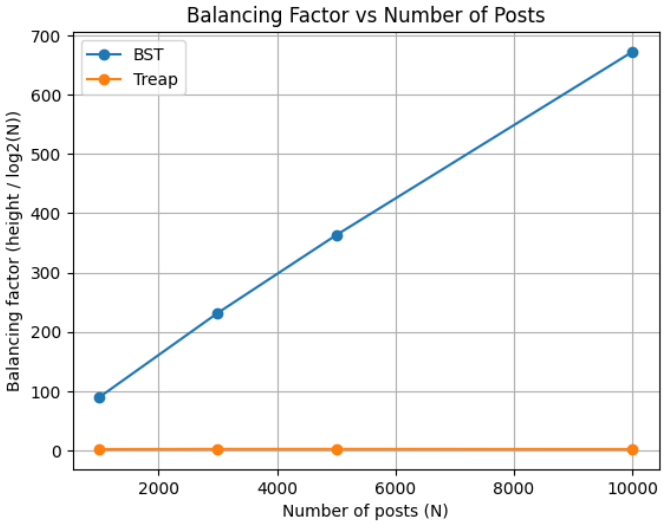
7. Visualization

Six plots were generated:

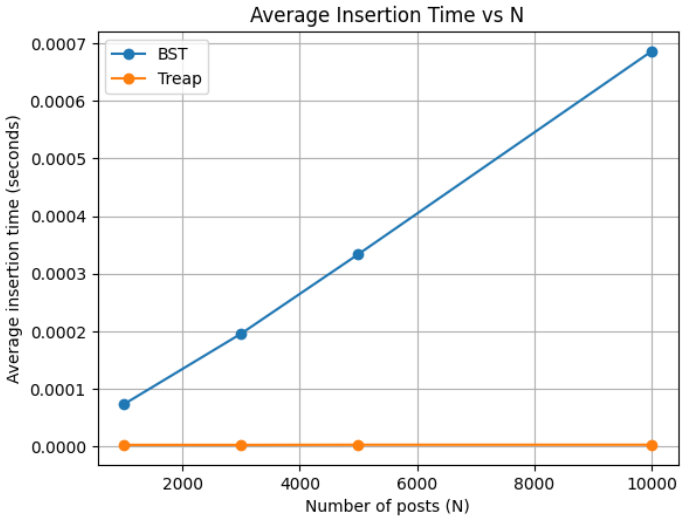
- Height vs N



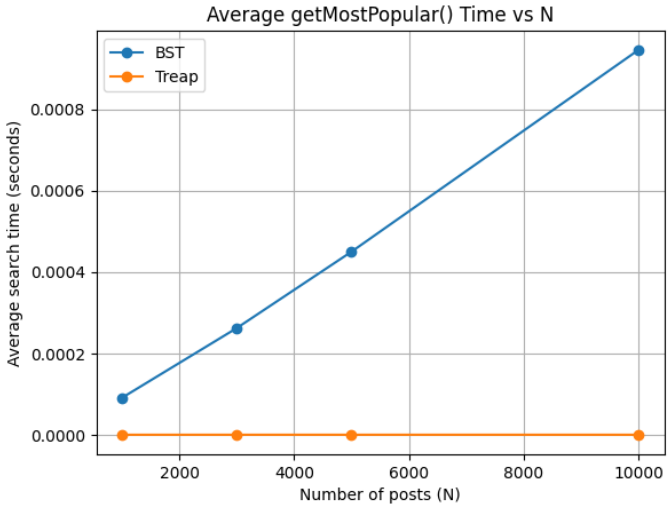
- Balancing factor vs N



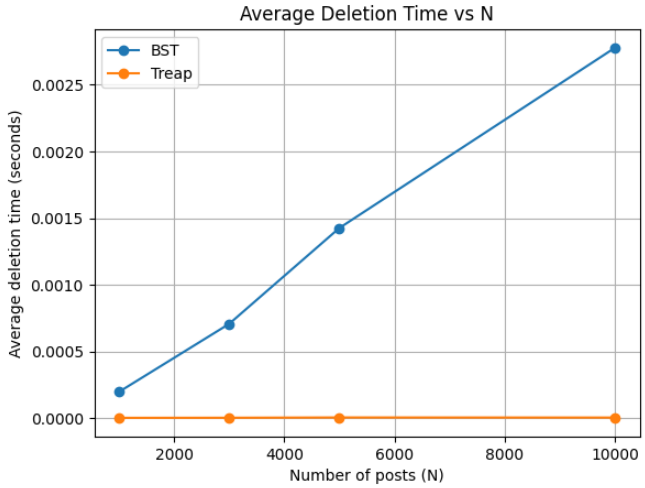
- Avg insertion time vs N



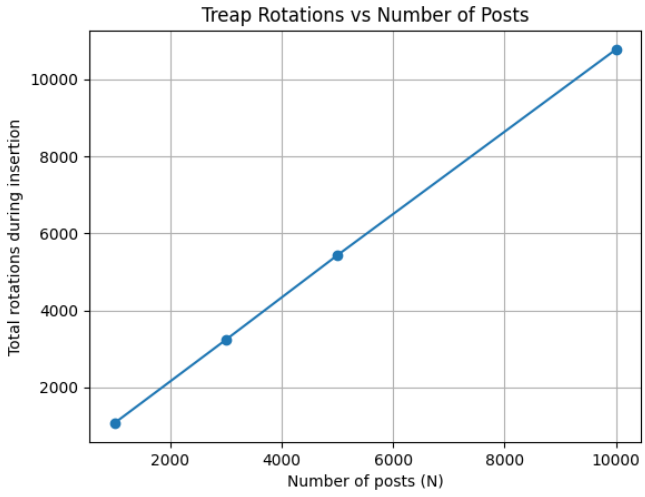
- Avg search time vs N



- Avg deletion time vs N



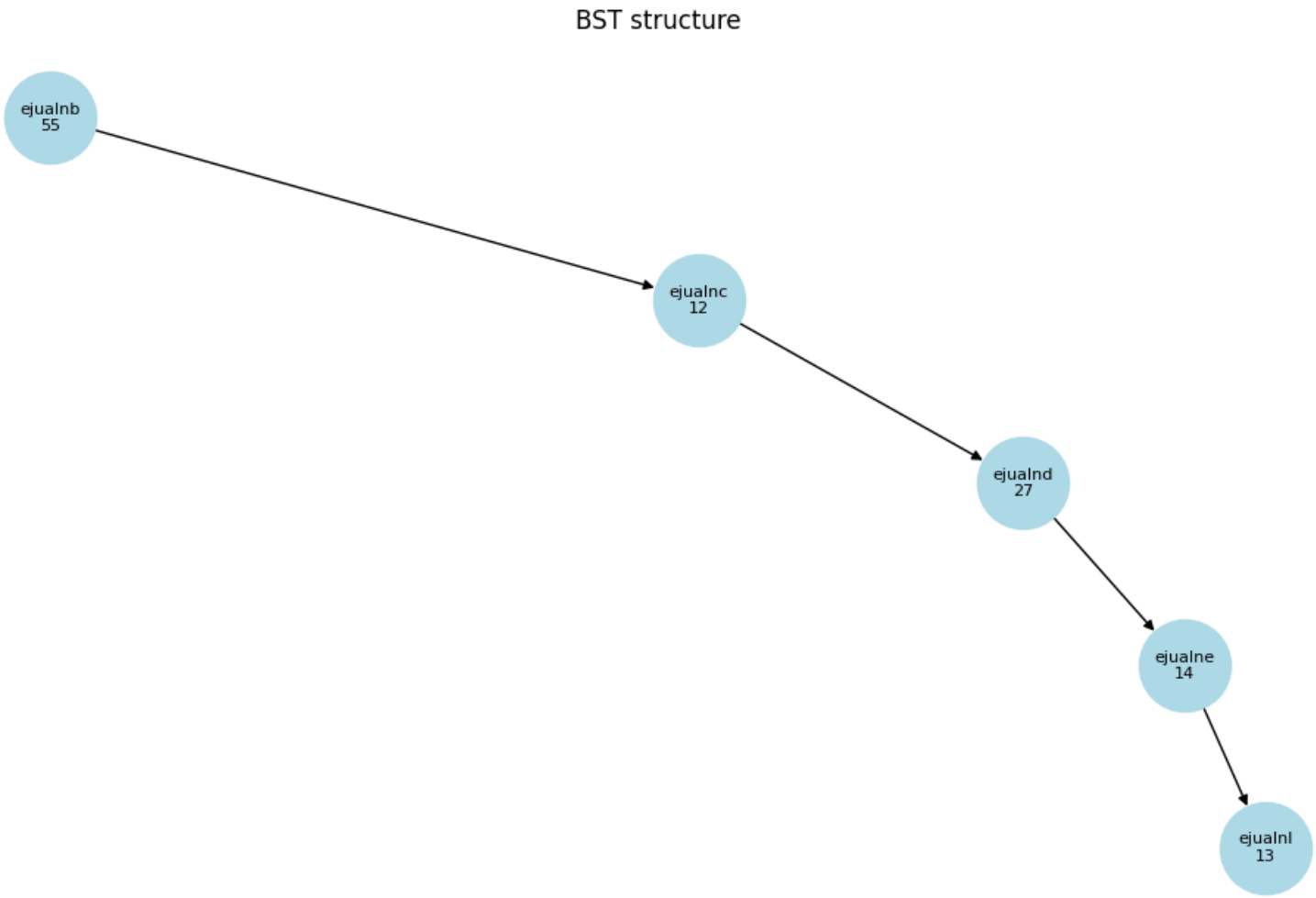
- Rotations vs N (treap only)

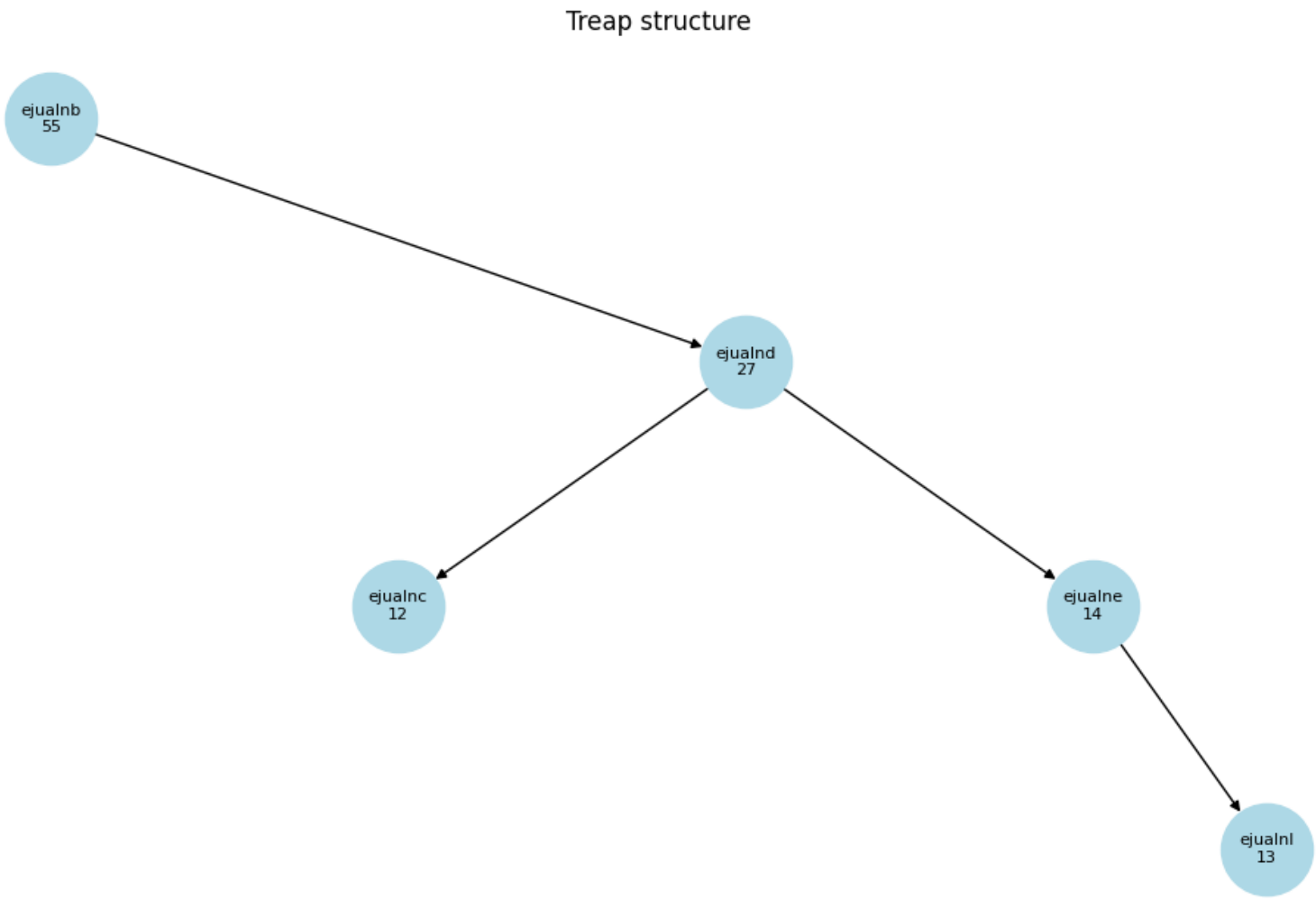


These demonstrate stark performance differences in favor of Treap under real-time update conditions.

8. Bonus Work

Visual tree rendering using NetworkX illustrated BST skewness vs Treap balance.





Treap Union was implemented by traversing and reinserting nodes from two treaps into a merged treap.

Merged most popular: ('b4', 204, 80)

Merged height: 6

Treap Intersection extracts common IDs to form a new treap.

Intersection root: None

9. Challenges

Handling massive compressed datasets with streaming constraints.
Correct treap rotations and deletion logic.Runtime limits within Kaggle environment.

10. Conclusion

Treaps significantly outperform standard BSTs for dynamic feed management tasks, maintaining near-logarithmic height, fast popularity retrieval, and reliable deletion while managing large streaming datasets. For production-scale systems where priority updates are frequent, Treaps offer substantial structural and performance benefits over unbalanced BSTs.

Operation Complexity Table (BST vs Treap)

Operation	BST Complexity	Treap Complexity (Expected)
addPost()	O(h) → O(N) worst	O(log N)
deletePost()	O(h) → O(N) worst	O(log N)
getMostPopular()	O(N)	O(1)
getMostRecent(k)	O(h + k)	O(log N + k)
likePost()	O(1)	O(log N)
Union / Intersection	Not supported	$O(m \cdot \log((n + m) / m))$